

Assignment 1: Foundations, Pointers & ADTs

Auqib Hamid Lone

Department of Computer Science & Engineering, GCET Ganderbal

August 17, 2026

Due: two weeks from issue. There is no automatic late penalty — if you need more time, ask before the deadline and an extension will be granted (see the course policy in the syllabus). Answer every question. Show all working for Numerical and Design questions. Each question tests one idea from Week 1 — read it fully before answering; every sub-part follows directly from the wording of the part before it.

1.1 Conceptual

Q1. A program is built from exactly three control patterns: **sequence**, **selection**, and **iteration**. Consider the function below.

```
int classify(int score) {  
    int grade = 0;           // line A  
    if (score >= 40)         // line B  
        grade = 1;  
    for (int i = 0; i < grade; i++) // line C  
        grade = grade + 0;  
    return grade;           // line D  
}
```

- (a) Which control pattern does line A illustrate?
- (b) Which control pattern does line B illustrate?
- (c) Which control pattern does line C illustrate?
- (d) The lecture's own example (`larger(int a, int b)`) treats a function's *parameter list* as an instance of sequence — the values simply arrive, one after the other, before anything else happens. Applying that same reading here: which control pattern does `classify`'s parameter list (`int score`), together with line D's `return grade;`, illustrate, and why?

Q2. A running C program keeps data in two different regions: the **stack** and the **heap**.

- (a) State, in one sentence each, what determines *when* a stack variable's storage is reclaimed, and what determines *when* a heap block's storage is reclaimed.
- (b) Give one scenario where a value must live on the heap because it would not survive on the stack.

- (c) Give one scenario where the stack is clearly the right choice, and say why using the heap there would be wasteful.

Q3. Distinguish between a **contract** statement and an **implementation** statement for the bag ADT introduced in class (operations `insert(x)`, `member(x)`, `size()`). For each of the following, say whether it is part of the contract or of one implementation, and justify in one sentence:

- (a) “`member(7)` answers yes or no.”
- (b) “The values are stored in an array.”
- (c) “`insert(x)` puts `x` into the bag.”
- (d) “`size()` reports the number of values present.”

1.2 Numerical

Q4. On a 64-bit machine, consider the C declaration

```
struct node {  
    int    data;  
    struct node *next;  
};
```

- (a) State how many bytes `sizeof(struct node)` reports, assuming `sizeof(int) == 4` and a pointer is 8 bytes, with no padding.
- (b) A program executes `struct node *p = malloc(sizeof(struct node));`. State what lives on the stack and what lives on the heap after this line, and what `p` itself holds.
- (c) If `malloc` returns `NULL`, what must the program do before it dereferences `p`, and why?

Q5. Consider the two independent requests below, each on a fresh, previously-unused region of heap memory:

```
int *a = malloc(3 * sizeof(int));  
int *b = calloc(3, sizeof(int));
```

- (a) State the content of the three integers at `a` immediately after the `malloc` call, before anything is written to them. Justify from the definition of `malloc`.
- (b) State the content of the three integers at `b` immediately after the `calloc` call, before anything is written to them. Justify from the definition of `calloc`.
- (c) Both calls request the same number of bytes. Name the one guarantee that distinguishes `calloc` from `malloc`, and state what it costs `calloc` to provide that guarantee.

Q6. A program holds `p = calloc(4, sizeof(int))`, and the four slots currently hold 5, 10, 15, 20. It then executes:

```
int *q = realloc(p, 7 * sizeof(int));  
if (q == NULL) return 1;  
p = q;
```

- (a) After this succeeds, state the values in slots 0–3 of the block `p` now points to, and justify from the definition of `realloc`.
- (b) State what can be said about the values in the three new slots (4–6), and justify — is this the same guarantee `calloc` gives, or a different one?
- (c) Suppose instead `realloc` had failed and returned `NULL`. Explain why the code checks `q` rather than reassigning `p` directly, and what would go wrong with the idiom `p = realloc(p, 7 * sizeof(int));` written as a single line with no check.

1.3 Design

Q7. Write a complete C function `create_node` that:

- (a) Takes an integer `data` as its only parameter.
- (b) Allocates one `struct node` (with fields `data` and `next`) on the heap using `malloc`.
- (c) Checks the result of `malloc` against `NULL` and returns `NULL` on failure (the caller can then decide how to recover).
- (d) Initialises both fields — `data` to the parameter, `next` to `NULL` — and returns the pointer.

State the function’s return type and write the body. Your solution must follow the pointer discipline from the lecture: check `malloc`, initialise every field, and use `p->field` form.

Q8. Write a complete C function

```
int *grow_scores(int *scores, int old_count, int new_count);
```

that manages a dynamically-sized array of exam scores, following the allocation discipline from the lecture. Your function must:

- (a) If `scores` is `NULL` (no array exists yet) and `old_count` is 0, allocate a fresh block for `new_count` integers using `calloc`, so that every slot starts at a well-defined value. Return `NULL` if this allocation fails.
- (b) Otherwise (an array already exists), use `realloc` to grow the existing block from `old_count` to `new_count` integers, following the check-before-overwrite idiom from Q6(c) — never assign a possibly-`NULL` result directly into `scores` before checking it. Return `NULL` if this reallocation fails, leaving the original block untouched.
- (c) On success, return the (possibly moved) pointer to the caller.

In one or two sentences beneath your function, state what — if anything — your function guarantees about the content of the newly added slots (index `old_count` through `new_count-1`) in each of the two branches (a) and (b), and explain why the two branches differ on this point.